Appendix B

Optional Topics for Experienced .NET Developers

This appendix collects useful topics that assume more experience than an absolute beginner is likely to have in *Chapter 1*. Release and support policies are especially relevant to teams maintaining production applications, while file-based apps and advanced editor features become easier to appreciate after you understand projects, types, methods, and object-oriented programming.

This appendix covers the following topics:

- Understanding .NET releases and support terms
- Running C# file-based apps
- Using editor navigation and inline hints

Understanding .NET releases and support terms

Enterprise .NET developers need to understand .NET releases and their support terms.

.NET versions are either **Long-Term Support** (**LTS**), **Standard-Term Support** (**STS**) (formerly known as **Current**), or **Preview**, as described in the following list:

- **LTS** releases are supported by Microsoft for 3 years after **General Availability** (**GA**), or 1 year after the next LTS release ships, whichever is longer.
- **STS** releases are supported by Microsoft for 2 years after GA, or 1 year after the next STS or LTS release ships, whichever is longer.
- **Preview** and **Release Candidate** (**RC**) releases are for public testing. These should only be used by adventurous programmers who want to live on the bleeding edge, or programming book authors who need to have early access to new features. Preview and RC releases are not usually supported by Microsoft but some may be declared **Go Live**, meaning they are supported by Microsoft in production. You must migrate to the GA release as soon as it is available to maintain support.

STS and LTS releases receive critical patches throughout their lifetime for security and reliability.



Good practice: You must stay up to date with the latest patches to get support. For example, if a system is running on .NET Runtime version 11.0.0 and then version 11.0.1 is released, you must install version 11.0.1 to get support. These updates are released on the second Tuesday of each month, aka **Patch Tuesday**.

Support durations

To better understand your choices of STS and LTS releases, it is helpful to see their support durations visualized, with 3-year-long black bars for LTS releases, and 2-year-long gray bars for STS releases, as shown in *Figure B.1*:

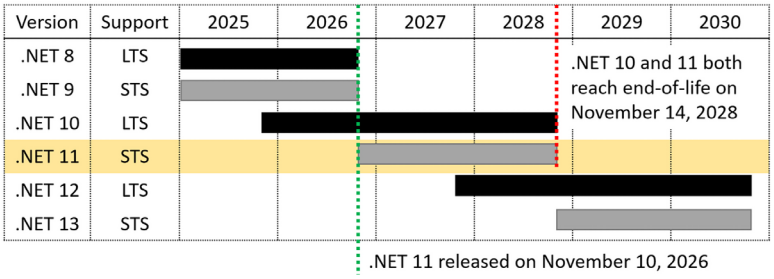


Figure B.1: Support durations for recent and planned STS and LTS releases

During the two-year lifetime of .NET 11, .NET 10 will still be supported, and .NET 12 will be released. I have tried to keep in mind that you might choose to use future versions like .NET 12 or .NET 13 with this book; however, obviously, the book cannot cover whatever their new features will be, since I don't know what they will be.

Actually, some .NET 12 features are already available in preview with .NET 11, like unsafe evolution, and I cover that in my book, *C, C++, and C# Programming for Beginners*, because that book goes deeper into memory management than this book, and is more academic leaning. You can read more about the unsafe evolution preview feature: `https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/proposals/unsafe-evolution`.

If you have existing projects that target .NET 9 or earlier, then they are now unsupported so you must migrate them to .NET 10 or .NET 11 as soon as possible. As soon as .NET 12 is released, start upgrading your .NET 10 and .NET 11 projects to it. You will have a year to do so before .NET 10 and .NET 11 reach their end of life on November 14, 2028.

.NET release quality

There is no difference in quality between an LTS and an STS release. LTS releases do not have fewer bugs than STS releases. The teams plan what features they want to add, implement them, and release them. STS releases are no more likely to include experimental features than LTS releases. Literally the only difference between LTS and STS releases is how long Microsoft promises to support the release.

Understanding .NET support phases

During the lifetime of a .NET version, it passes through several phases, during which it has varying levels of support, as described in the following list, and as shown in *Figure B.2*:

- **Preview**: These are not supported at all. .NET 11 Preview 1 to Preview 7 were in this support phase from February 2026 to August 2026.
- **Release Candidate (RC)**: Supported if they have **Go Live** status until GA, and then they become immediately unsupported. You must upgrade to the GA release as soon as it is available. .NET 11 RC1 and RC2 were in this support phase in September and October 2026, respectively.
- **Active**: Full support with monthly releases for bug and security fixes. You must update to any patch version release every month to have support. .NET 11 will be in this support phase from November 2026 to May 2028.
- **Maintenance**: Supported but it only gets security fixes for the last six months of its lifetime. .NET 11 will be in this support phase from May 2028 to November 2028. In May 2028, .NET 10 and .NET 11 both enter their maintenance phase. At that point if you have existing projects that target .NET 10 or .NET 11, then you should migrate those projects to .NET 12.
- **End of life (EOL)**: Not supported. .NET 10 and .NET 11 will both reach their EOL on November 14, 2028.

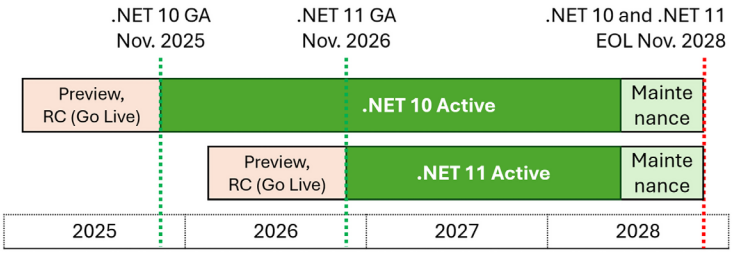


Figure B.2: .NET support phases from preview to maintenance

Understanding end of life

End of support or **end of life** (**EOL**) means the date after which bug fixes, security updates, or technical assistance are no longer available from Microsoft.

At the time of publishing on November 10, 2026, all versions of modern .NET have reached their EOL, except those shown in the following list:

- .NET 10 and .NET 11 will be in their active phase until May 2028, then they will be in their maintenance phase for six months until they reach EOL on November 14, 2028.
- .NET 12 will be available in preview from February 2027, be active from November 9, 2027 until its maintenance phase in May 2030, and finally it will reach EOL in November 2030.



Check which .NET versions are currently supported and when they will reach EOL: `https://github.com/dotnet/core/blob/main/releases.md`.

When .NET 10 and .NET 11 reach their end of support on November 14, 2028, you can expect the following:

- Projects that use .NET 10 or .NET 11 will continue to run
- No new security updates will be issued for .NET 10 or .NET 11, and therefore, continuing to use an unsupported version will increasingly expose you to security vulnerabilities
- You might not be able to access technical support for any .NET 10 or .NET 11 applications that you continue to use
- You will get `NETSDK1138` build warnings when targeting .NET 10 or .NET 11
- You will get warnings in Visual Studio when targeting .NET 10 or .NET 11



Prompt: `Please explain the difference between LTS, STS, Preview, Go Live, Active, Maintenance, and End of Life support for .NET. Give practical advice for choosing a version for learning and for production.`

Understanding .NET Runtime and .NET SDK versions

If you have not built a standalone app, then the .NET Runtime is the minimum you need to install so that an operating system can run a .NET application. The .NET SDK includes the .NET Runtime, as well as the compilers and other tools needed to build .NET code and apps.

.NET Runtime versioning follows semantic versioning; that is, a major increment indicates breaking changes, minor increments indicate new features, and patch increments indicate bug fixes.

.NET SDK versioning does not follow semantic versioning. The major and minor version numbers are tied to the runtime version they are matched with. The third number follows a convention that indicates the minor and patch versions of the SDK. The third number starts at 100 for the initial version (equivalent to 0.0 for the minor and patch numbers). The first digit increments with new SDK feature bands, and the other two digits increment with patches within that band.

You can see examples of this in *Table B.1*:

Change	.NET Runtime	.NET SDK
Initial release	11.0.0	11.0.100
SDK bug fix	11.0.0	11.0.101
Runtime and SDK bug fix	11.0.1	11.0.102
New SDK feature band	11.0.1	11.0.200

Table B.1: Examples of changes and versions for a .NET Runtime and .NET SDK



Prompt: `Please explain the difference between the .NET Runtime version and the .NET SDK version. Why does the SDK version not use semantic versioning in the same way as the runtime?`

Listing and removing versions of .NET

All future versions of the .NET runtime are compatible with its major version. For example, if a project targets `net11.0`, then you can upgrade the .NET Runtime to future versions like `11.0.1`, `11.0.2`, and so on. In fact, you *must* upgrade the .NET Runtime every month to maintain support.

All future versions of the .NET SDK maintain the ability to build projects that target previous versions of the runtime. For example, if a project targets `net11.0` and you initially build it using .NET SDK 11.0.100, then you can upgrade the .NET SDK to future bug-fix versions, like 11.0.101, or a major version, like 12.0.100, and that SDK can still build the project for the older targeted version.

A newer SDK can normally build projects that target earlier .NET versions. Before removing an older SDK, like 10.0.100, after you've installed .NET SDK 11 or later, check for repositories that use `global.json`, SDK-specific workloads, CI configurations, or tools that require that version.

You can see which SDKs and runtimes are currently installed using the following commands:

```
dotnet --list-sdks
dotnet --list-runtimes
dotnet --info
```



Good practice: To make it easier to enter commands at the command prompt or terminal, the following link lists all commands throughout the book that can be easily copied and pasted: `https://github.com/markjprice/cs15net11/blob/main/docs/command-lines.md`.

On Windows, use the **Apps & features** section to remove .NET SDKs. You can also use the "remove out of support components" option in Visual Studio to clean up old .NET SDKs from existing installations.

On Linux, there is no single mechanism: `https://learn.microsoft.com/en-us/dotnet/core/install/remove-runtime-sdk-versions?pivots=os-linux`.



You could use a third-party tool like Dots, "the friendly .NET SDK manager": `https://johnnys.news/2023/01/Dots-a-dotnet-SDK-manager`. But you must build the app from source on its GitHub repository, so I only recommend that for advanced developers.

Running C# file-based apps

As you have seen, as well as `.cs` code files, C# projects usually need a `.csproj` project file. The project file configures which version of the .NET Runtime you want to target, language features like nullability and global namespace imports, and references to other projects and packages.

However, when you are first learning a programming language, you probably won't change many of those options, so it's extra complexity without any benefit.

Rival languages like Python allow you to execute a code file without a project file.

You create a Python code file like `app.py`:

```
print("Hello, Python")
```

And then run the code file using the `python` **command-line interface (CLI)**:

```
python app.py
```

Introducing C# file-based apps

C# has a similar feature named **file-based apps** that allows developers to execute single `.cs` files directly:

```
dotnet run app.cs
```

Microsoft's official terms are **file-based app** versus **project-based app**. This feature is also sometimes referred to as `dotnet run app.cs`, which is misleading because you can actually miss out the `run` part of the command and it still runs.

A file-based app uses top-level statements and an implicit project. It lets you run a short C# program before learning about project files. It is also convenient for experiments and small scripts.

Creating a C# file-based app

File-based apps are not supported in Visual Studio because it's a command-line feature.

Let's see an example:

1. In the `Chapter01` directory, create a new directory named `NoProject`.
2. In the `NoProject` directory, use a plain text editor to create a file named `hello.cs` that writes a message to the console:

   ```
   Console.WriteLine("Hello C# with no project file!");
   ```

3. At the command prompt or terminal, in the `NoProject` folder, use the `dotnet` CLI to run the `hello.cs` file:

   ```
   dotnet hello.cs
   ```

4. Note the result:

   ```
   Hello C# with no project file!
   ```



Prompt: `Please explain why a simple C# console app can now be run as a single file without a project file. Compare this with running a Python script.`

Configuring no-project C# code files

To enhance functionality, you can use `#:` directives at the top of the `.cs` file.

Most `#:` directives are allowed to appear as duplicates across included files when their values match (`#:sdk`, `#:property`, `#:package`) which allows self-contained library files that declare their own dependencies without conflicting when other files also include them.

Specifying the SDK

You can specify the SDK, like switching to `Microsoft.NET.Sdk.Web` for ASP.NET Core projects:

```
#:sdk Microsoft.NET.Sdk.Web
```

To separate the SDK ID and an explicit version number, use an `@`, for example, `#:sdk Aspire.AppHost.Sdk@9.5.0`.

Setting MSBuild properties

You can set MSBuild properties, like setting the C# language version to `preview`:

```
#:property LangVersion=preview
```

Specifying commands on Linux

On Linux, you can specify `#!` aka "shebang" commands:

```
#!/usr/bin/env dotnet
```

This allows you to execute the `.cs` file without prefixing it with the `dotnet run` command. You must also mark the `.cs` file as executable in the filesystem. For example, at the command prompt or terminal: `./hello.cs`

Adding NuGet package references

You can add NuGet package references like `Humanizer` and then use its functionality:

```
#:package Humanizer@2.14.1

using Humanizer;

Console.WriteLine(TimeSpan.FromDays(1).Humanize());
```

Adding project references

You can add references to project-based apps:

```
#:project ../MyClassLib/MyClassLib.csproj
```

You can reference full project files with `#:project`, but there are other ways to split file-based apps across ordinary `.cs` files without first creating a project.

Splitting file-based apps across multiple files

In the simplest examples so far, a file-based app has everything in one `.cs` file. One file is enough for a lesson, an experiment, or a small script. As the program grows, separating the code can make it easier to navigate.

File-based apps also support splitting across multiple files without first creating a full project. The `#:include` directive allows a file-based app to include other source files, for example, helper files and model files:

1. In the `NoProject` folder, create a file named `helpers.cs`:

   ```
   static class Helpers
   {
     public static string FormatCustomer(Customer customer)
     {
       return $"{customer.Name} has joined.";
     }
   }
   ```

2. In the `NoProject` folder, create a folder named `Models`, and inside it, create a file named `Customer.cs`:

   ```
   public record Customer(string Name);
   ```

3. In the `NoProject` folder, create a file named `app.cs`:

   ```
   #!/usr/bin/env dotnet
   #:include helpers.cs
   #:include Models/Customer.cs

   Customer customer = new("Bob");

   Console.WriteLine(Helpers.FormatCustomer(customer));
   ```

4. At the command prompt or terminal, in the `NoProject` folder, run the app:

   ```
   dotnet app.cs
   ```

5. Note the output:

```
Bob has joined.
```

The app now contains several source files but still has no `.csproj` file. The `app.cs` file is still the entry point. The other files are brought into the file-based app because `app.cs` includes them using the `#:include` directive.



Although `#!` directives like `#!/usr/bin/env dotnet` are ignored on Windows, we must include one to prevent a `CA2266` warning: `https://learn.microsoft.com/en-us/dotnet/fundamentals/code-analysis/quality-rules/ca2266`.



Good practice: Use `#:include` for source files that naturally belong to the same small app, for example, a helper class, a `record` type, or a few related utility methods. If the app keeps growing, convert it to a full project.

The `#:include` directive can also include compiled DLL references, so you can reference prebuilt libraries directly:

```
#:include ./libs/MyLibrary.dll

MyLibrary.Helper.DoWork();
```

Referencing another C# file as a library

File-based apps also support the `#:ref` directive. Instead of including another file as part of the same app using `#:include`, `#:ref` references another file-based app as a library. This is a way for file-based apps to split code across files without first creating a project, and transitive references are supported.

1. In the `NoProject` folder, create a file named `lib.cs`:

```
#:property OutputType=Library

namespace MyLib;

public static class Greeter
{
  public static string Greet(string name)
  {
    return $"Hello, {name}!";
  }
}
```

2. In the `NoProject` folder, create a file named `appwithref.cs`:

```
#:property ExperimentalFileBasedProgramEnableRefDirective=true
#:ref lib.cs

Console.WriteLine(MyLib.Greeter.Greet("File-based reference"));
```

3. Run the app:

```
dotnet appwithref.cs
```

4. The output will be:

```
Hello, File-based reference!
```

Do not confuse `#:ref` with `#:project`. The `#:ref` directive references another C# file-based app. The `#:project` directive references a project file, such as `../MyClassLib/MyClassLib.csproj`.



Good practice: Use `#:include` when you want another source file to be treated as part of the same file-based app. Use `#:ref` when you want to reference another file-based app as a library. Use `#:project` when you want to reference a normal project file, such as a `.csproj`.

Converting file-based apps to a full project

You can later convert a file-based app into a project-based app:

```
dotnet project convert app.cs
```

By default, this will create a project-based app in a new subfolder with the name of the main file. For example, for `app.cs` it will create a folder named `app` containing all the new files.

Publishing file-based apps

You can publish a file-based app as a native executable:

```
dotnet publish app.cs
```

By default, it is published as a native-compiled AOT app in an `artifacts\<appname>` subfolder. You can disable that by setting a property at the top of the file:

```
#:property PublishAot=false
```

To learn more about the file-based app feature: `https://devblogs.microsoft.com/dotnet/announcing-dotnet-run-app/`.

To view a video demonstration: `https://www.youtube.com/watch?v=98MizuB7i-w`.



Good practice: Keep file-based apps small. Splitting code across files is useful, but once you need tests, multiple packages, detailed build settings, publishing options, or several developers working together, a full project is usually the better choice.

You have now run C# in both project-based and file-based forms. The repository contains complete project-based solutions, optional material, and corrections that can help when your results differ from the book.

Using editor navigation and inline hints

This section is all about how to find quality information about C# and .NET programming.

Getting definitions of types and their members

Two of the most useful features of a code editor are **Go To Definition** (*F12*) and **Go To Implementation** (*Ctrl + F12*):

- **Go To Definition** opens the declaration of a type or member. Depending on the editor and its settings, that declaration might come from source code, reference metadata, or decompiled code.
- **Go To Implementation** is most useful for interfaces, abstract members, and inherited members. It finds the classes or members that provide concrete implementations.

Some tools, such as ILSpy .NET Decompiler, will even reverse-engineer the metadata and IL code back into C# or another language for you.



Warning! In Visual Studio, **Go To Definition** should go to the decompiled metadata for a member or type. But if you have previously viewed the source link, then it goes to that. **Go To Implementation** should go to the source link implementation for a member or type. But if you have disabled the source link, then it goes to the decompiled metadata. In VS Code, if you try to use the **Go To Definition** feature and you get a `Request textDocument/definition failed` error, then navigate to **Settings** | **Extensions** | **C#** | **Text Editor**, and clear the **Enable navigation to source link and embedded sources** checkbox.

Let's see how to use the **Go To Definition** feature:

1. In your preferred code editor, open the solution/folder named `Chapter01`.

   If you are using Visual Studio, follow these steps:

   - Navigate to **Tools** | **Options**.
   - In the search box, enter `navigation to source`.
   - Click the first **Advanced** link, as shown in *Figure B.3*. (The second one is for Visual Basic.)
   - Note that you are taken to **Text Editor** | **C#** | **Advanced** in the legacy **Options** dialog box.

- Clear the **Enable navigation to Source Link and Embedded sources** checkbox, and then click **OK**, as shown in *Figure B.3*:

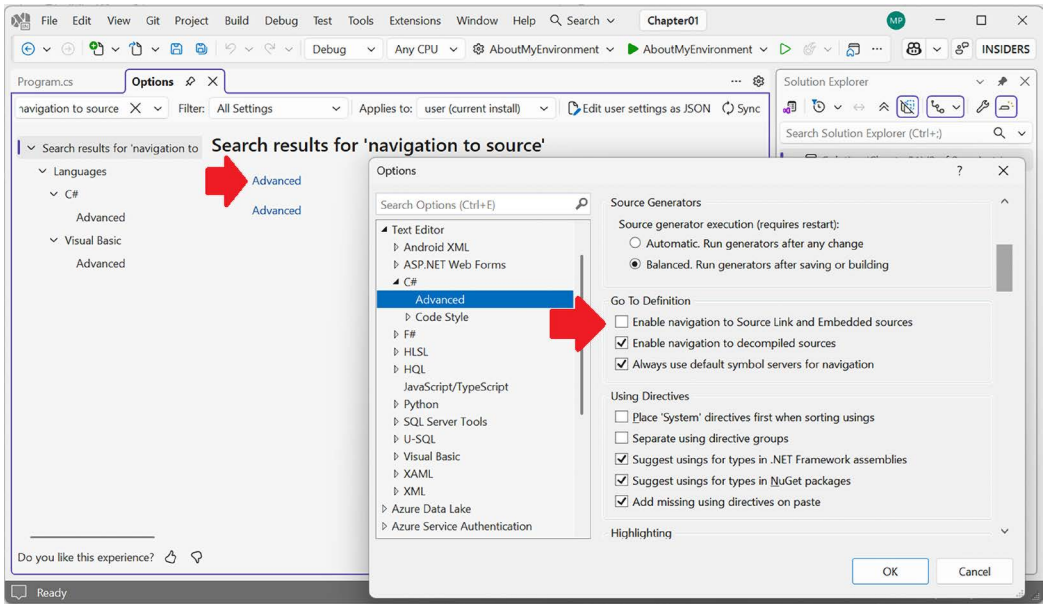


Figure B.3: Disabling Source Link for the Go To Definition feature in Visual Studio



Definitions can be either reverse engineered from metadata or loaded from the original source code if that is enabled. Personally, I find the code from metadata more useful, as you are about to see. At the end of this section, try switching the **Source Link** option back on to see the difference.

2. In the `HelloCS` project, at the bottom of `Program.cs`, enter the following statement to declare an integer variable named `z`:

```
int z;
```

3. Right-click on **int**, and then choose **Go To Definition** in Visual Studio or VS Code. In Rider, choose **Go to | Go to Declaration or Usages**.
4. In the code window that appears, you can see how the `int` data type is defined:

```
#region Assembly System.Runtime, Version=11.0.0.0, Culture=neutral,
PublicKeyToken=b03f5f7f11d50a3a
// C:\Program Files\dotnet\packs\Microsoft.NETCore.App.Ref\11.0.0\ref\
net11.0\System.Runtime.dll
#endregion

#nullable enable
```

```
using System.Diagnostics.CodeAnalysis;
using System.Globalization;
using System.Numerics;
using System.Runtime.CompilerServices;

namespace System
{
  public readonly struct Int32 : IComparable,
    IComparable<Int32>, IConvertible, ...
  {
    public const Int32 MaxValue = 2147483647;
    public const Int32 MinValue = -2147483648;
    ...
  }
}
```

You can see that `int`:

- Is defined using the `struct` keyword
- Is in the `System.Runtime` assembly
- Is in the `System` namespace
- Is named `Int32`
- Is, therefore, an alias for the `System.Int32` type
- Implements interfaces such as `IComparable`
- Has constant values for its maximum (`2147483647`) and minimum (`-2147483648`) values
- Has methods such as `Parse` (not visible in the preceding code)



Currently, the **Go To Definition** feature is not that useful to you because you do not yet know what all of this information means. By the end of the first part of this book, which consists of *Chapters 2* to *6* and teaches you about the C# language, you will know enough for this feature to become very handy.

5. In the code editor window, scroll down to find the `Parse` method with a single `string` parameter:

```
public static Int32 Parse(string s)
```

6. Expand the code and review the comments that document this method, as shown in *Figure B.4*:

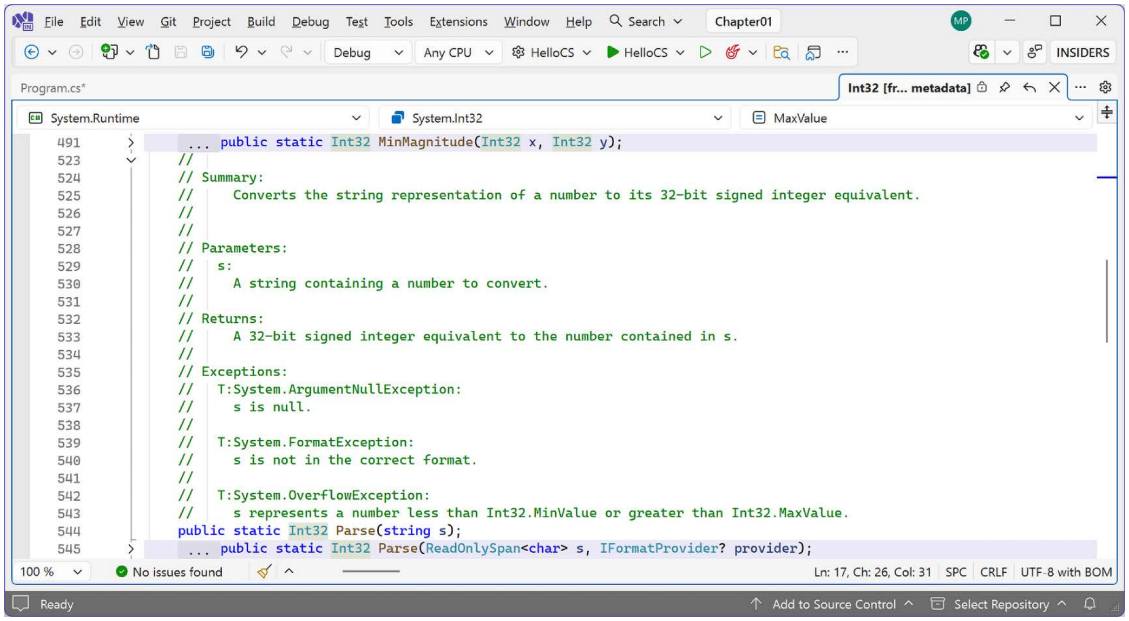


Figure B.4: The comments for the Parse method with a single string parameter

In the comments, you will see that Microsoft has documented the following:

- A summary that describes the method.
- Parameters like the `string` value that can be passed to the method.
- The return value of the method, including its data type.
- Three exceptions that might occur if you call this method, including `ArgumentNullException`, `FormatException`, and `OverflowException`. Now, we know that we could choose to wrap a call to this method in a `try` statement and which exceptions to catch. For expected user input, `int.TryParse` is normally preferable because invalid text is not exceptional. `Parse` is useful when invalid input should be treated as an error.



Prompt: `Please explain why int in C# is really System.Int32. What does it mean that Int32 is a struct, has MaxValue and MinValue, and implements interfaces?`

Configuring inline aka inlay hints

Throughout the code in this book, when calling a method, I often explicitly specify named parameters to help you learn what is going on. For example, I have specified the names of the `format` and `arg0` parameters in the following code:

```
Console.WriteLine(format: "Value is {0}.", arg0: 19.8);
```

Inline hints, aka **inlay hints**, show the names of parameters without you having to type them, as shown in *Figure B.5*:

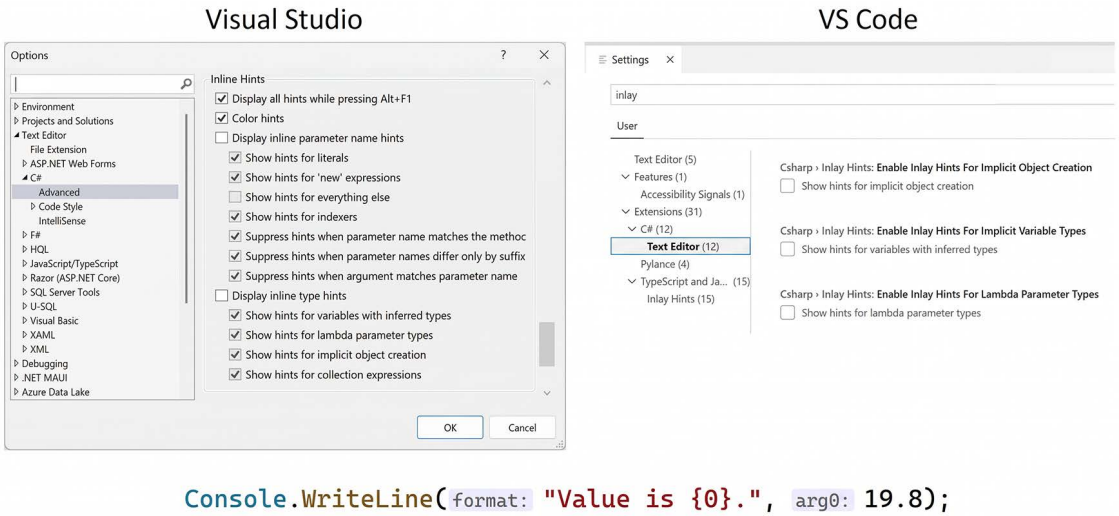


Figure B.5: Configuring inline hints, aka inlay hints

Most code editors have this feature, which you can enable permanently or only when a key combination like *Alt + F1* or *Ctrl* is held down:

- In Visual Studio, navigate to **Tools** | **Options**, then **Text Editor** | **C#** | **Advanced**, scroll down to the **Inline Hints** section, select the checkboxes that you want, and then click **OK**
- In VS Code, navigate to **File** | **Preferences** | **Settings**, search for `inlay`, select the **C#** filter, and then select the checkboxes that you want
- In Rider, in **Settings**, navigate to **Editor** | **Inlay Hints** | **C#** | **Parameter Name Hints**, and then select the checkboxes that you want



Prompt: `Please explain inline hints, also called inlay hints, in C# editors. When are they useful for learning, and when might they become visual noise?`

This is the end of Appendix B.